PEARLS AQI PREDICTOR

Final Project Report

Karachi Air Quality Index Forecasting System

10Pearls SHINE Internship — Data Sciences Track

Prepared by
Umer Khan

September 2026

Final system: Random Forest + Hopsworks + GitHub Actions + FastAPI + Render + Streamlit

Table of Contents

List of Figures

Executive Summary

Pearls AQI Predictor is an end-to-end machine learning and MLOps system developed to forecast Karachi's Air Quality Index (AQI) and present the results through a publicly accessible dashboard. The system combines historical environmental data collection, cleaning, feature engineering, cloud feature management with Hopsworks, Random Forest regression, model registration, scheduled GitHub Actions workflows, a FastAPI prediction service, recursive 72-hour forecasting, and SHAP explainability. Historical hourly AQI and weather data were collected for a single Karachi coordinate, producing more than 2,000 records. The final production model uses 29 engineered features and a 300-tree Random Forest Regressor. On a chronological held-out test set for next-hour prediction, the model achieved MAE 0.530, RMSE 0.714, and R^2 0.994. Longer-horizon validation showed the expected degradation of recursive forecasting, with MAE increasing to 6.10 at 24 hours, 10.00 at 48 hours, and 11.29 at 72 hours. These results are intentionally reported separately so that the strong next-hour performance is not confused with 72-hour accuracy. The final system demonstrates the complete lifecycle from data acquisition to deployment while also documenting practical challenges involving environment compatibility, Feature Store materialization, dependency management, live data freshness, and recursive forecast uncertainty.

Key Final Results

Item	Final Result
Historical records	>2,000 hourly records
Usable modelling rows	2,284
Train / test	1,827 / 457 chronological rows
Engineered features	29
Production model	Random Forest Regressor, 300 trees
MAE / RMSE / R^2	0.530 / 0.714 / 0.994
24h validation	MAE 6.10, RMSE 7.20, R^2 0.549
48h validation	MAE 10.00, RMSE 11.97, R^2 -0.175
72h validation	MAE 11.29, RMSE 12.93, R^2 -0.286
Model Registry	karachi_aqi_next_hour_rf, Version 2
Feature Group	karachi_aqi_features, Version 4
Feature View	karachi_aqi_fv, Version 1
Automation	Hourly feature + daily training + hourly 72h forecast

1. Introduction

Air quality is an important environmental and public-health problem for large urban areas. Karachi is a particularly relevant setting because pollution conditions can change throughout the day and are influenced by both pollutant concentrations and weather. A useful forecasting system therefore needs to do more than display a current AQI value: it must combine historical behavior, current environmental variables, engineered time-series features, and a reproducible machine-learning workflow.

The Pearls AQI Predictor was developed as an internship project for the 10Pearls SHINE Internship — Data Sciences Track. The objective was to demonstrate an end-to-end data-science and MLOps workflow rather than producing only a standalone model or notebook. The completed system covers historical data acquisition, preprocessing, feature engineering, cloud Feature Store integration, model training and evaluation, Model Registry management, automated workflows, an API, recursive multi-day forecasting, explainability, and a public dashboard.

The final architecture uses managed cloud services and scheduled automation. GitHub Actions handles recurring workflows, Hopsworks provides feature and model management, Render hosts the FastAPI prediction service, and Streamlit Community Cloud hosts the interactive dashboard. This design reduces the amount of infrastructure that must be manually maintained while keeping the major stages of the machine-learning lifecycle separated and reproducible.

2. Problem Statement

The project addresses the problem of forecasting Karachi's AQI for the next hour and extending that model recursively to a 72-hour forecast. The central challenge is that air quality is a time-dependent phenomenon. The AQI at one hour is usually related to recent AQI values, pollutant concentrations, weather conditions, and recurring time patterns.

The system therefore treats forecasting as a supervised regression problem. At time t, the engineered feature vector represents the current environmental state and recent historical context. The production target is the AQI at time t+1. Once the next-hour model has been trained, its output can be fed back as an input to generate later hours, producing a 72-step recursive forecast.

This formulation has an important consequence: the production model is directly validated for next-hour prediction, while the 24-, 48-, and 72-hour outputs are evaluated separately because recursive error can accumulate as the forecast horizon increases.

3. Project Objectives

The main objectives were:

- Build an end-to-end AQI forecasting pipeline for Karachi.
- Collect and process historical hourly AQI, pollutant, and weather data.
- Engineer meaningful time-series features, including lags, rolling statistics, and AQI changes.
- Store engineered features in Hopsworks Feature Store.
- Train and evaluate machine-learning forecasting models using a chronological split.
- Register the production model in Hopsworks Model Registry.
- Automate feature updates, model training, and forecast regeneration using GitHub Actions.
- Provide a FastAPI prediction service for current and next-hour AQI.
- Generate a recursive 72-hour forecast.
- Build a Streamlit dashboard showing current conditions, forecasts, model performance, and explanations.
- Incorporate SHAP-based explainability.
- Document limitations honestly, especially the difference between next-hour validation and longer-horizon recursive forecasting.

4. Requirements and Scope

The project scope was designed around the internship requirements: external environmental data, historical backfilling, feature engineering, Feature Store integration, model training and evaluation, Model Registry, automation, an interactive dashboard, explainability, AQI health categories, and multi-day forecasting.

The implementation satisfies these requirements through a combination of Python scripts, Hopsworks, GitHub Actions, FastAPI, Render, and Streamlit. The final implementation also includes a dedicated forecast workflow, so the forecast file used by the dashboard can be regenerated automatically rather than depending only on manual execution.

The system should be understood as a practical internship-scale forecasting platform rather than a city-wide regulatory monitoring system. It currently represents Karachi using a single geographic coordinate and uses an Open-Meteo AQI estimate in the live prediction path. These limitations are important when interpreting the results.

5. System Architecture

The system is organized as a sequence of data, feature, model, serving, and presentation layers.

External data sources → Data collection/backfill → Cleaning and feature engineering → Hopsworks Feature Store → Training and evaluation → Hopsworks Model Registry → FastAPI prediction service → Streamlit dashboard.

In parallel, GitHub Actions schedules the operational workflows. The hourly feature workflow updates data and features. The daily training workflow trains and registers the production model. The hourly forecast workflow regenerates the 72-hour forecast CSV and commits an updated artifact to the repository.

The architecture separates training data management from live serving. Hopsworks is used as the cloud feature-management layer for the training pipeline. The live API constructs the current feature row using current Open-Meteo data and locally available historical AQI context. This separation was introduced after encountering Feature Store materialization delays during development and makes the live endpoint less dependent on the immediate availability of newly inserted Feature Store rows.

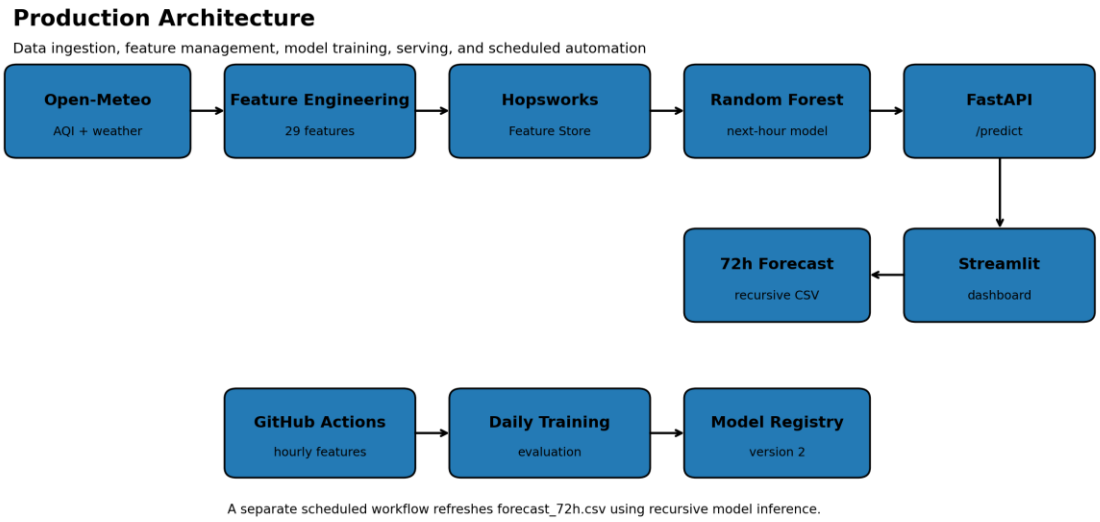


Figure 14. End-to-end production architecture linking data, features, model management, serving, forecasting, and automation.

6. Data Sources and Data Collection

Two environmental APIs were explored during development: AQICN and Open-Meteo. Open-Meteo was especially useful for historical backfilling because it provides hourly air-quality and weather variables for a specified geographic coordinate.

The historical dataset was collected for Karachi using latitude 24.8607 and longitude 67.0011. The backfill process collected AQI, PM2.5, PM10, ozone, nitrogen dioxide, sulfur dioxide, carbon monoxide, temperature, humidity, atmospheric pressure, and wind speed. The resulting local database contains more than 2,000 hourly records spanning multiple months.

AQICN was also investigated for live/station-based air-quality information. During final deployment, the generic Karachi AQICN feed was not sufficiently reliable as a current observation source because the returned timestamp could be stale. Consequently, the deployed live prediction path uses Open-Meteo current AQI and weather data. The dashboard and API describe this appropriately as a current AQI estimate rather than presenting it as a direct physical monitoring-station measurement.

This source decision is an important part of the project's engineering story: the data source was not selected only because it was available, but because the final application needed a timestamped and usable live input.

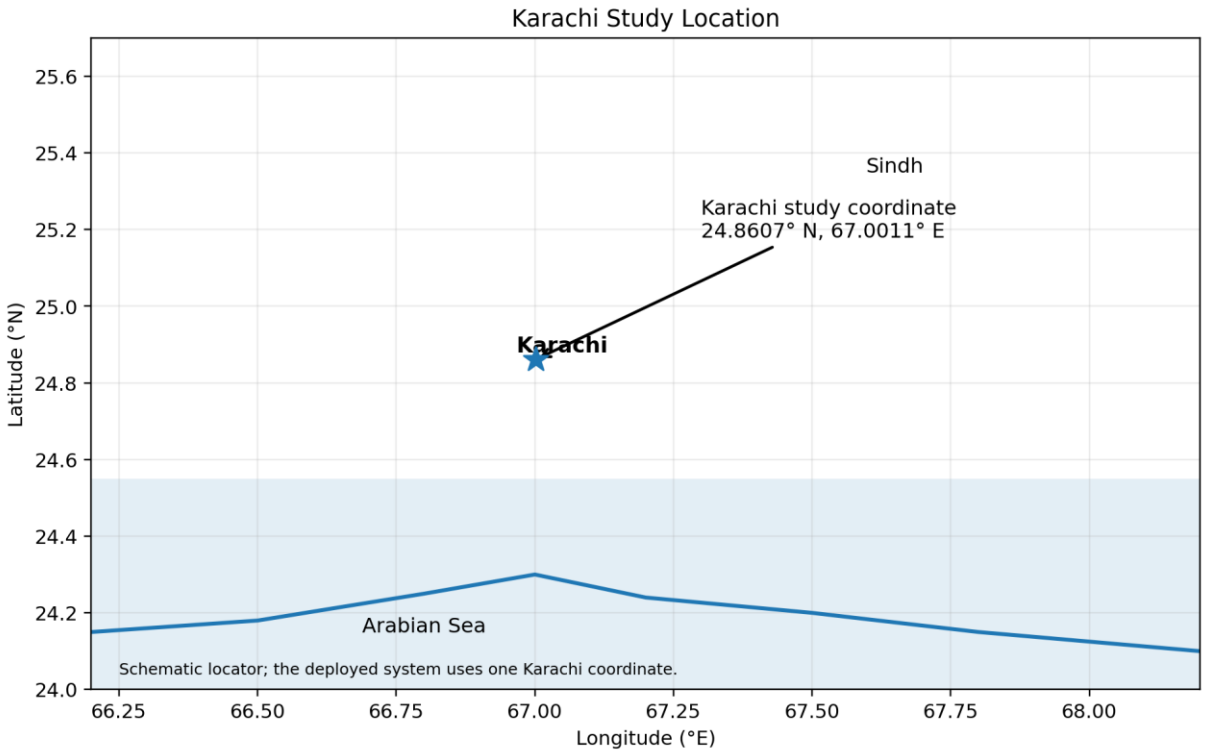


Figure 13. Karachi study location represented by the single coordinate used for environmental data.

7. Data Cleaning and Preprocessing

Raw environmental API responses are not assumed to be immediately suitable for modelling. The pipeline validates timestamps, orders the historical records chronologically, checks missing values, and ensures sufficient historical context exists before feature construction.

Several engineered features require prior observations. For example, a 24-hour lag cannot be calculated for the earliest rows, and a 24-hour rolling statistic requires enough preceding observations. Rather than filling these values with arbitrary numbers, rows without sufficient context are removed from the modelling dataset. The same principle is applied to the target: a row used for supervised learning must have a valid future AQI value.

This approach reduces the risk of introducing artificial information into the model and keeps the training data aligned with the actual forecasting problem.

8. Exploratory Analysis and Time-Series Reasoning

The project's modelling strategy was informed by the temporal behavior of AQI. AQI tends to be persistent over short time intervals, meaning that recent AQI values contain substantial information about the next hour. This observation motivated the inclusion of multiple lag features and rolling statistics.

The dataset also contains environmental variables that provide additional context. Pollutants such as PM2.5 and PM10 describe particulate pollution, while ozone, NO2, SO2, and CO provide additional atmospheric information. Temperature, humidity, pressure, and wind speed provide weather context that may influence pollutant dispersion and concentration.

For the final report, the exploratory analysis should be illustrated using the project's existing plots where available, especially the actual-vs-predicted plot, residual plot, AQI time-series visualization, and feature-importance/SHAP plots. These figures make the numerical evaluation easier to interpret.

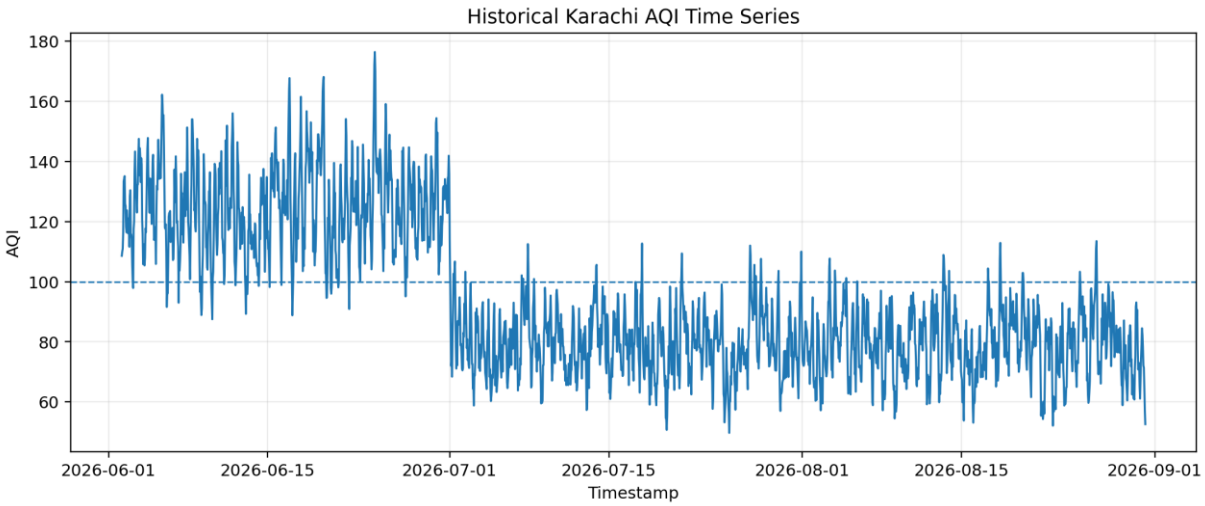


Figure 10. Historical Karachi AQI time series used to establish temporal context.

9. Feature Engineering

The final model uses 29 engineered features.

Calendar features: hour, day, month, day of week, and weekend indicator. These provide recurring temporal context.

AQI lag features: 1-hour, 3-hour, 6-hour, 12-hour, and 24-hour lags. The 1-hour lag is the most influential individual feature in the production Random Forest, which is consistent with short-term AQI persistence.

Rolling statistics: 3-, 6-, 12-, and 24-hour AQI rolling means, plus 6- and 24-hour rolling standard deviations. These describe recent level and volatility.

AQI change features: 1-hour, 6-hour, and 24-hour AQI changes. These describe whether air quality is stable, increasing, or decreasing over different time windows.

Pollutant features: PM2.5, PM10, ozone, NO2, SO2, and CO.

Weather features: temperature, humidity, atmospheric pressure, and wind speed.

Together, these features combine instantaneous environmental conditions with temporal context. This is particularly important for a one-hour-ahead model because a purely instantaneous feature set would ignore the strong persistence of AQI.

10. Model Development and Selection

Several modelling approaches were considered during development, including Ridge Regression, Random Forest, Gradient Boosting, and XGBoost, with broader experimentation also informed by the internship requirement to investigate different modelling approaches. The final production model was selected using both predictive performance and engineering considerations.

The final production model is a Random Forest Regressor configured with 300 trees, random state 42, and parallel processing. Random Forest was appropriate for the project because it can model nonlinear relationships, works well with mixed engineered features, requires relatively little feature scaling, and is practical for scheduled retraining.

The final production model is registered in Hopsworks Model Registry as karachi_aqi_next_hour_rf, Version 2. The model is trained specifically for next-hour AQI prediction. It is not claimed that the same validation metrics represent 72-hour forecast accuracy.

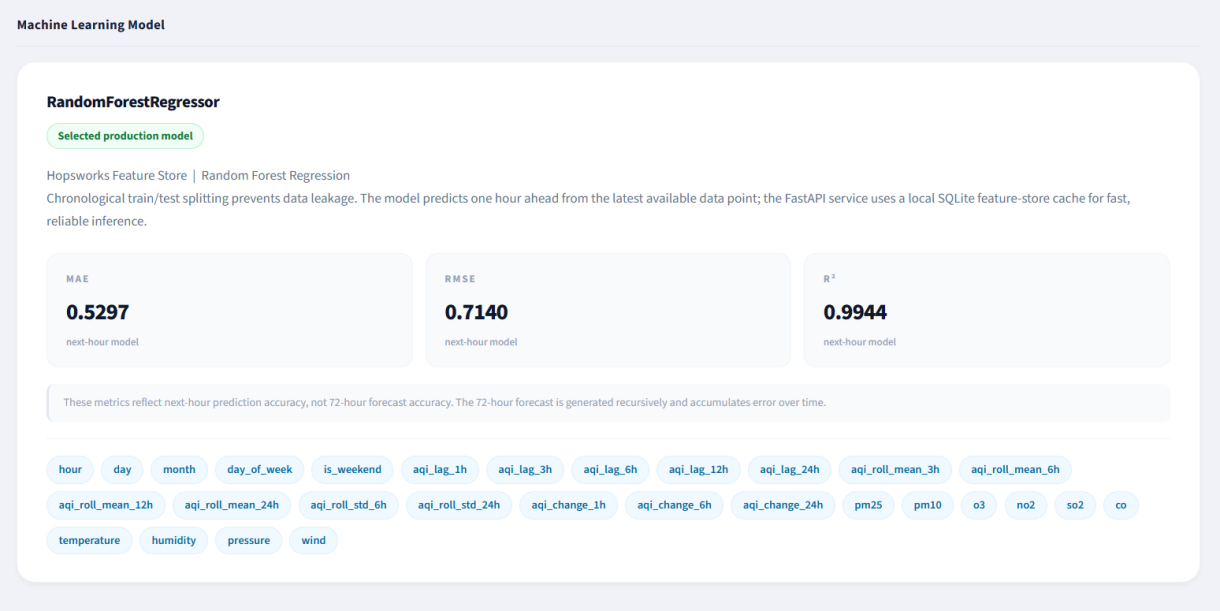


Figure 4. Production Random Forest configuration and next-hour model metrics.

11. Training Strategy and Leakage Prevention

Time-series forecasting requires special care when splitting data. Randomly shuffling hourly observations can allow future patterns to influence training and can produce an unrealistically optimistic evaluation.

The production pipeline therefore uses a chronological split. The final modelling dataset contains 2,284 usable rows. The first 1,827 rows are used for training and the most recent 457 rows are held out for testing. The ordering is preserved throughout the split.

The target is constructed as the AQI one hour ahead. In conceptual terms, the model learns:

Features at time t → AQI at time t+1.

This formulation makes the evaluation directly relevant to the deployed next-hour prediction task and avoids using future target values as input features.

12. Production Model Evaluation

The final Random Forest achieved the following results on the chronological held-out test set:

MAE = 0.530
RMSE = 0.714
R^2 = 0.994

MAE measures the average absolute prediction error in AQI units. RMSE gives greater weight to larger errors. R^2 describes the proportion of variance explained relative to a baseline based on the mean target.

The high R^2 should be interpreted in context. The previous-hour AQI is extremely influential because AQI is persistent over short time intervals. Feature-importance analysis confirms that aqi_lag_1h dominates the production model. Therefore, the next-hour result demonstrates strong short-term predictive performance, but it should not be interpreted as evidence that the system can maintain the same accuracy for a three-day recursive forecast.

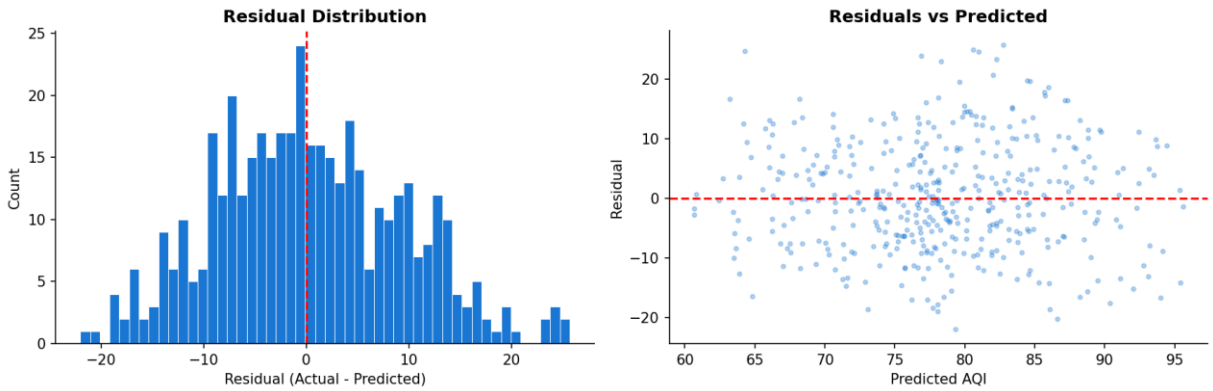


Figure 7. Residual distribution and residuals versus predicted AQI.

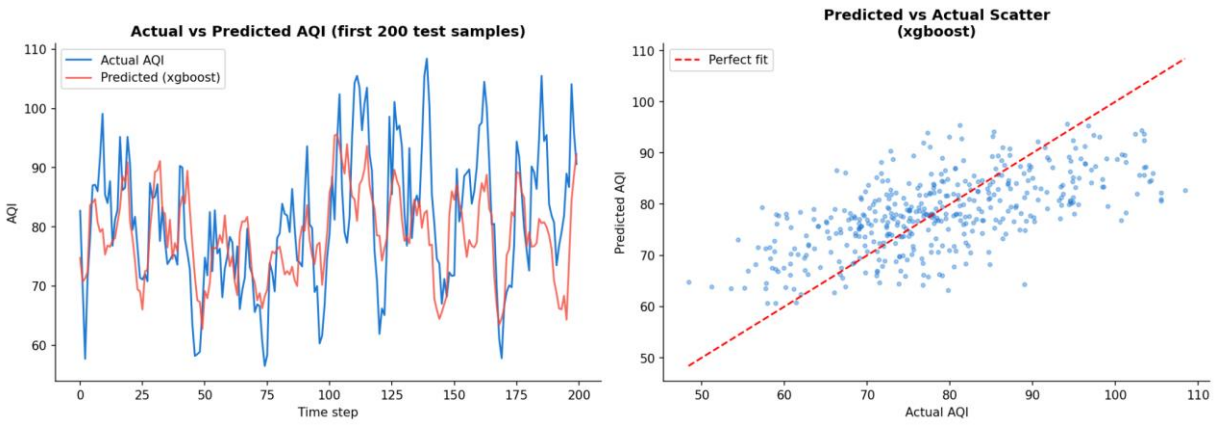


Figure 6. Actual versus predicted AQI on the held-out test samples.

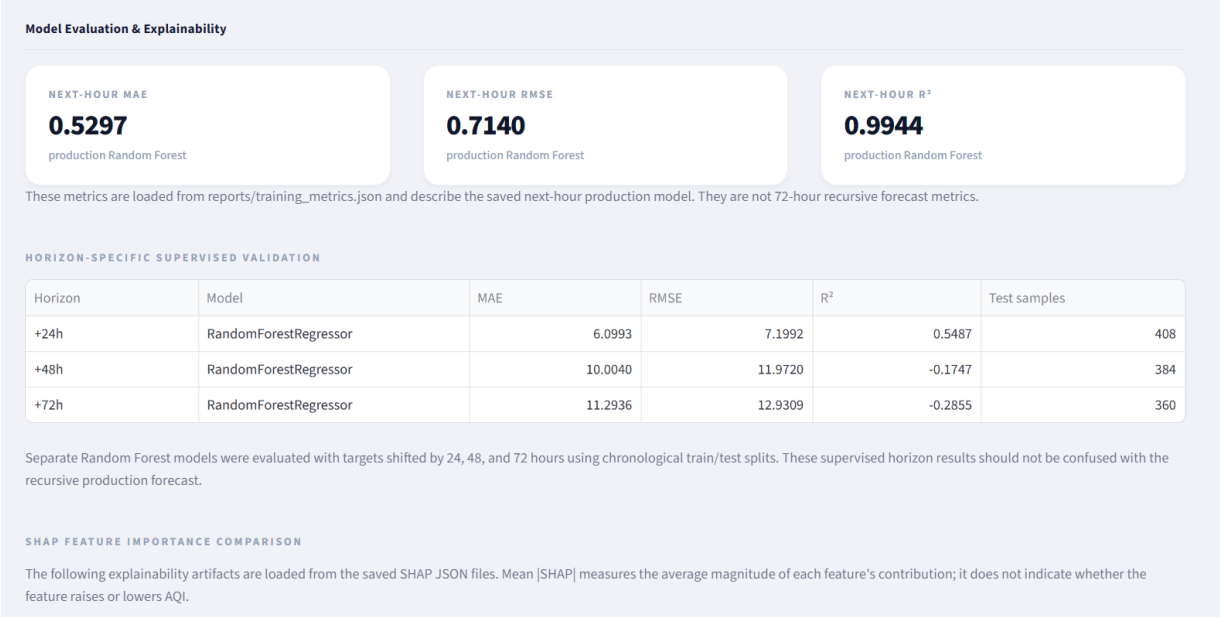


Figure 5. Production metrics and horizon-specific validation results.

13. Longer-Horizon Forecasting and Validation

The internship requires a three-day forecast, while the production model is trained for one-hour-ahead prediction. To bridge this gap, the system uses recursive forecasting. The first prediction is generated from the current feature state. That prediction is then used as the previous AQI value for the next step, and the process continues until 72 forecast points are produced.

The approach is operationally simple and allows the existing production model to generate a multi-day sequence. However, recursive forecasting introduces error propagation. Every future prediction depends partly on the preceding prediction, so small errors can accumulate.

Independent longer-horizon validation produced:

24 hours: MAE 6.10, RMSE 7.20, R^2 0.549.
48 hours: MAE 10.00, RMSE 11.97, R^2 -0.175.
72 hours: MAE 11.29, RMSE 12.93, R^2 -0.286.

The degradation is an important result rather than a failure to hide. It demonstrates why dedicated multi-horizon models would be a valuable future improvement. The dashboard therefore separates the validated next-hour model metrics from the longer-horizon validation results.

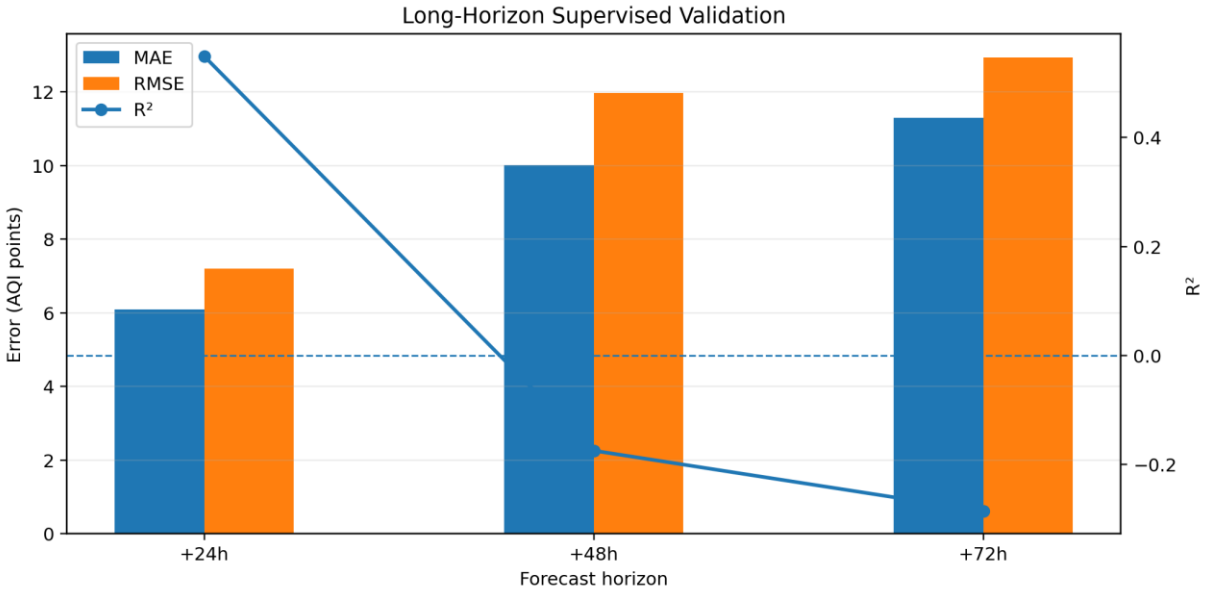


Figure 12. Supervised validation of Random Forest models at 24-, 48-, and 72-hour horizons.

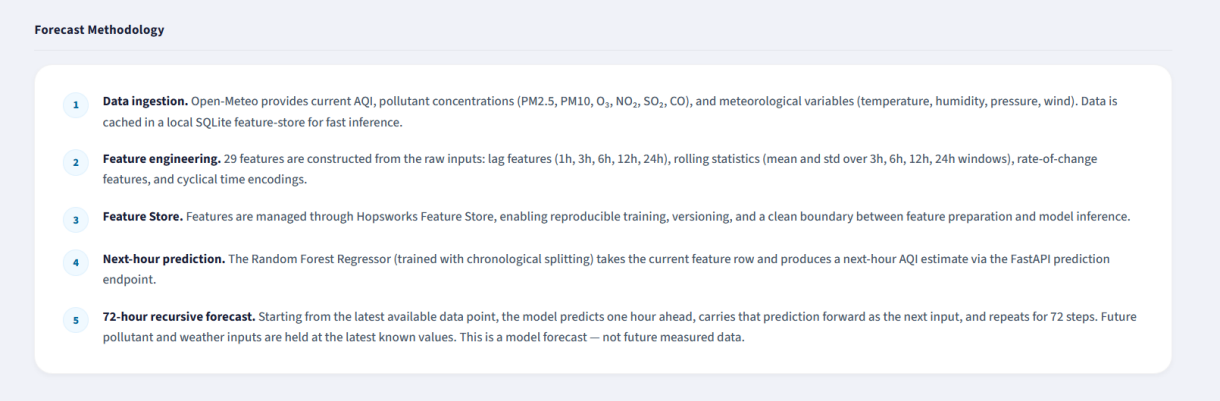


Figure 9. Recursive 72-hour forecasting methodology used by the forecasting pipeline.

14. Hopsworks Feature Store and Model Registry

Hopsworks is used as the cloud feature-management layer. The project uses the Hopsworks project pearls_aqi_predictors, Feature Group karachi_aqi_features, and Feature View karachi_aqi_fv.

The Feature Store provides a reproducible location for engineered training features rather than requiring the training process to reconstruct everything from raw API responses. The Feature View provides the training dataset consumed by the model pipeline.

The trained production model is registered as karachi_aqi_next_hour_rf, Version 2. The registry stores the production model as a versioned artifact together with evaluation information and training-data linkage. This supports model lifecycle management and makes later version comparison or rollback possible.

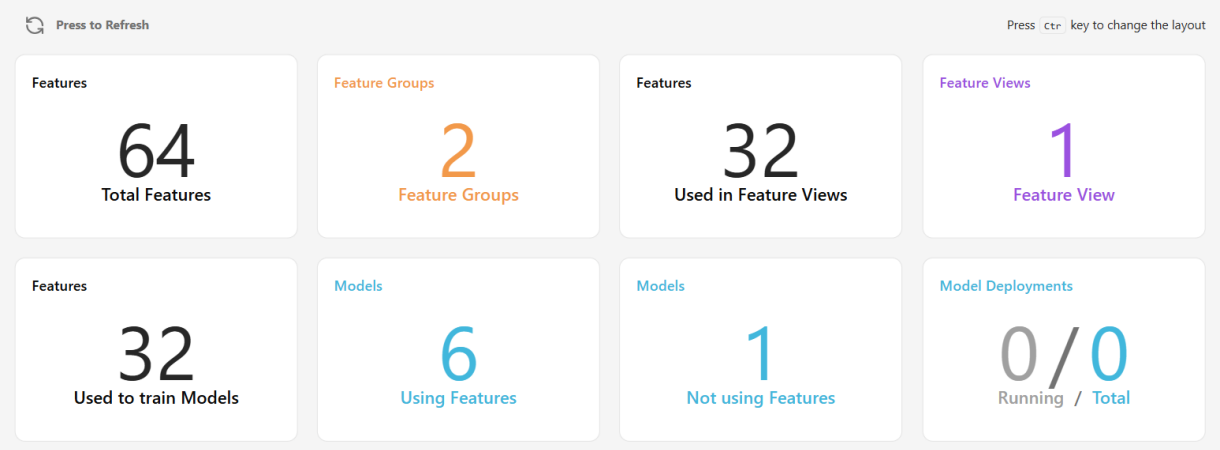


Figure 11. Hopsworks project overview showing Feature Groups, Feature Views, and model usage.

15. Automated MLOps Workflows

The final project contains three scheduled GitHub Actions workflows.

1. Hourly Feature Pipeline — fetches environmental data, processes it, creates engineered features, and updates the Feature Store.

2. Daily Training Pipeline — retrieves the historical training dataset, trains and evaluates the model, and registers the resulting model version in Hopsworks.

3. Hourly 72-Hour Forecast Pipeline — generates the recursive 72-hour forecast and commits the updated forecast CSV to the repository.

The workflows remove repeated manual execution from the normal operating cycle. The forecast workflow was specifically added so that the dashboard's 72-hour forecast artifact can be regenerated automatically on a schedule.

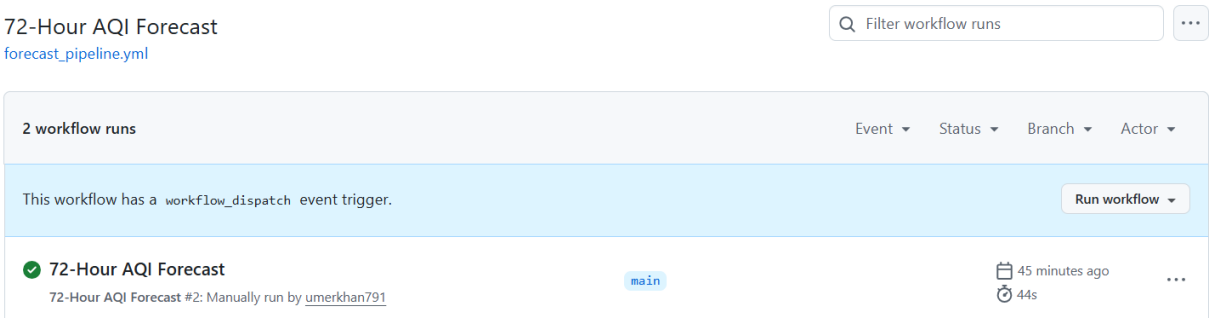


Figure 17. Successful 72-Hour AQI Forecast workflow run in GitHub Actions.

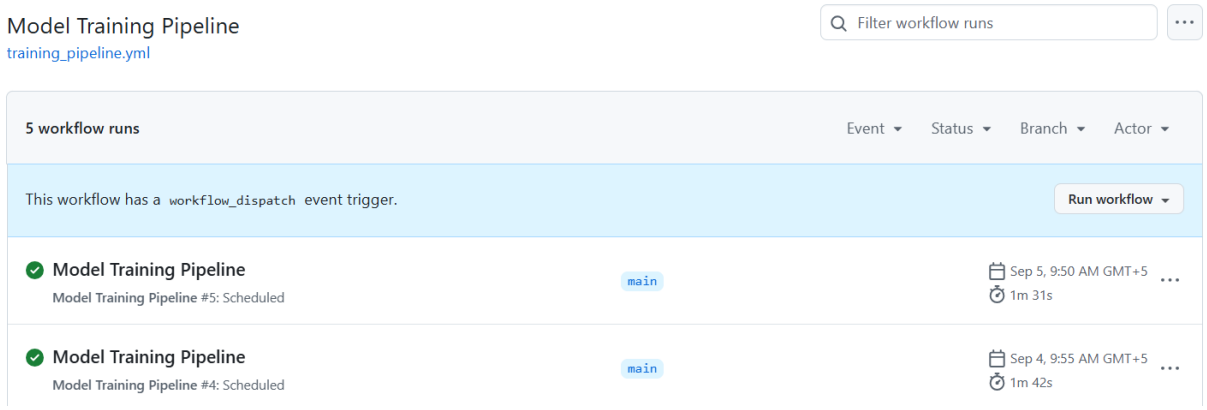


Figure 16. Successful daily Model Training Pipeline runs in GitHub Actions.

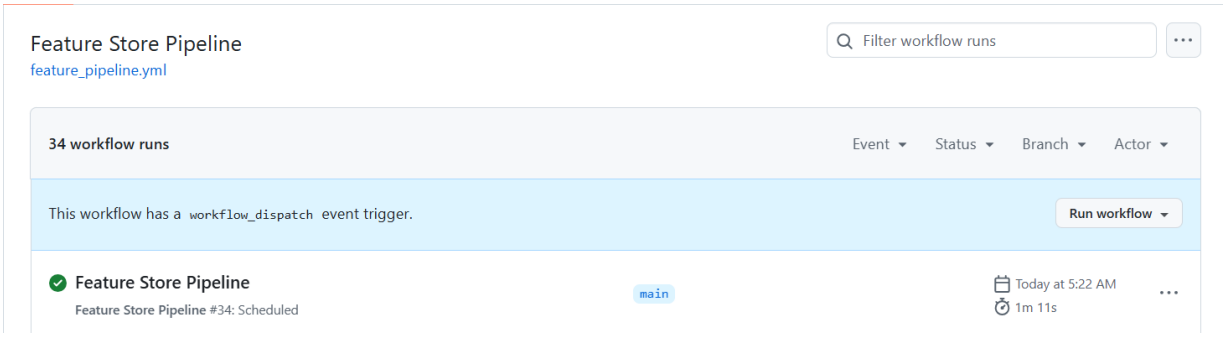


Figure 15. Successful hourly Feature Store Pipeline run in GitHub Actions.

16. Prediction API

A FastAPI service provides the application-facing prediction endpoint. The deployed API loads the Random Forest model, obtains current environmental information, constructs the required feature vector, performs the next-hour prediction, and returns the current AQI estimate, health category, forecast value, prediction timestamp, pollutants, weather conditions, and model metrics.

The API is hosted on Render and is consumed by the public Streamlit dashboard. The live serving path was intentionally designed to avoid dependence on immediate Feature Store materialization. This was a practical response to a development issue in which recently inserted Feature Store rows were not always immediately queryable.

The API therefore acts as the bridge between the machine-learning model and the user-facing application while keeping the dashboard independent of the model's internal Python execution environment.

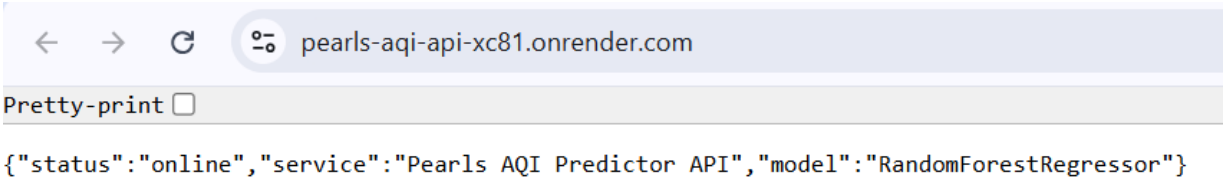


Figure 18. Public FastAPI service status confirming the deployed prediction backend is online.

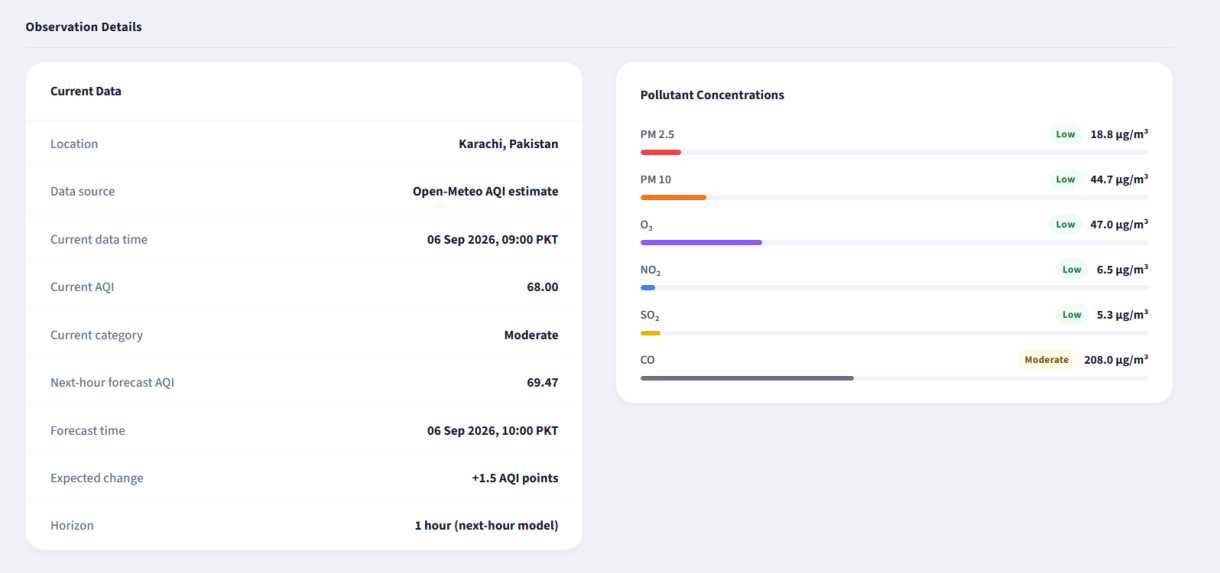


Figure 3. Current AQI, forecast, weather, and pollutant details exposed by the application.

17. Streamlit Dashboard

The Streamlit dashboard is the primary user-facing interface. It provides a visual summary of current AQI, the next-hour forecast, weather and pollutant conditions, the 72-hour recursive forecast, model evaluation, and SHAP explainability.

The dashboard includes:

- Current AQI estimate and health category.
- Next-hour predicted AQI and category.
- Pollutant readings for PM2.5, PM10, ozone, NO2, SO2, and CO.
- Weather readings including temperature, humidity, pressure, and wind.
- 72-hour forecast chart and hourly table.
- Day 1, Day 2, and Day 3 forecast summaries.
- Next-hour production metrics.
- Longer-horizon validation metrics.
- SHAP explainability.
- AQI health-category reference.
- Visual highlighting for elevated/hazardous conditions.

The dashboard is deployed through Streamlit Community Cloud and obtains current prediction data from the public FastAPI service.

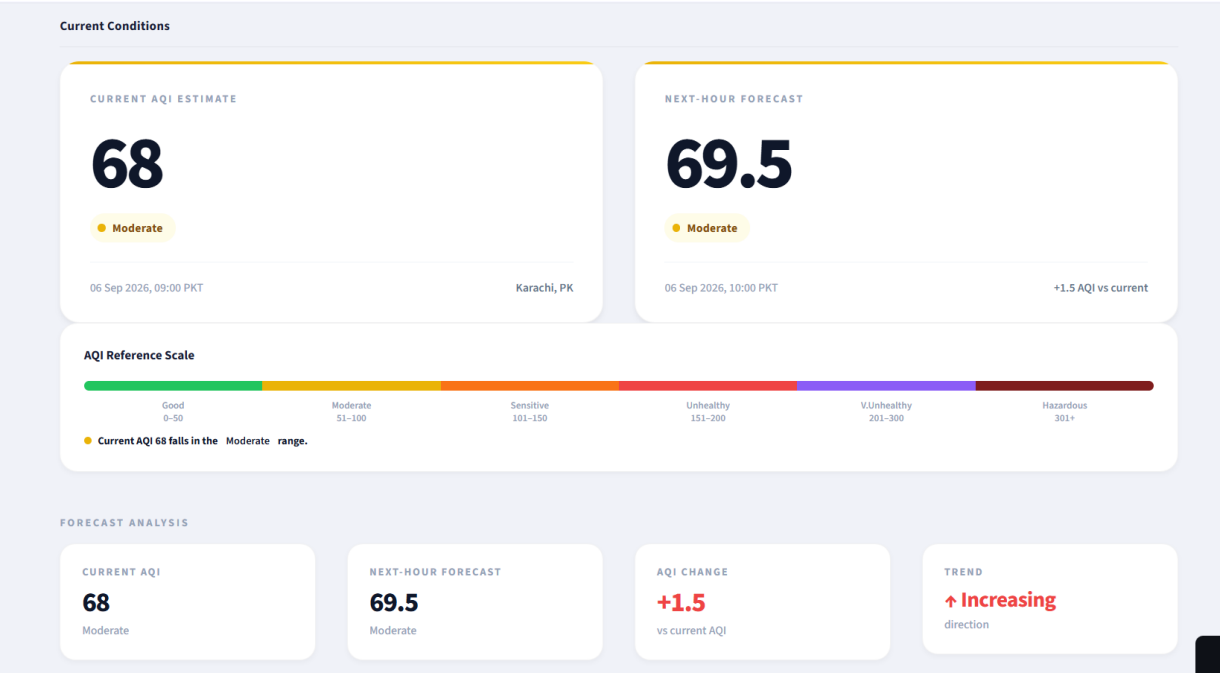


Figure 2. Dashboard current conditions, AQI scale, and next-hour forecast.

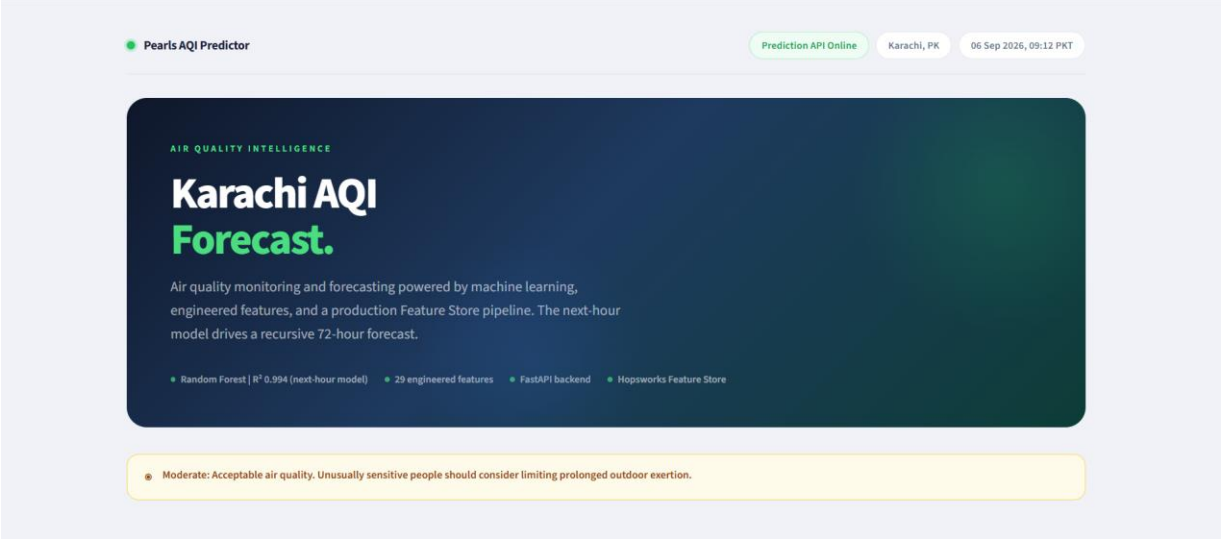


Figure 1. Deployed Streamlit dashboard overview for Karachi AQI monitoring and forecasting.

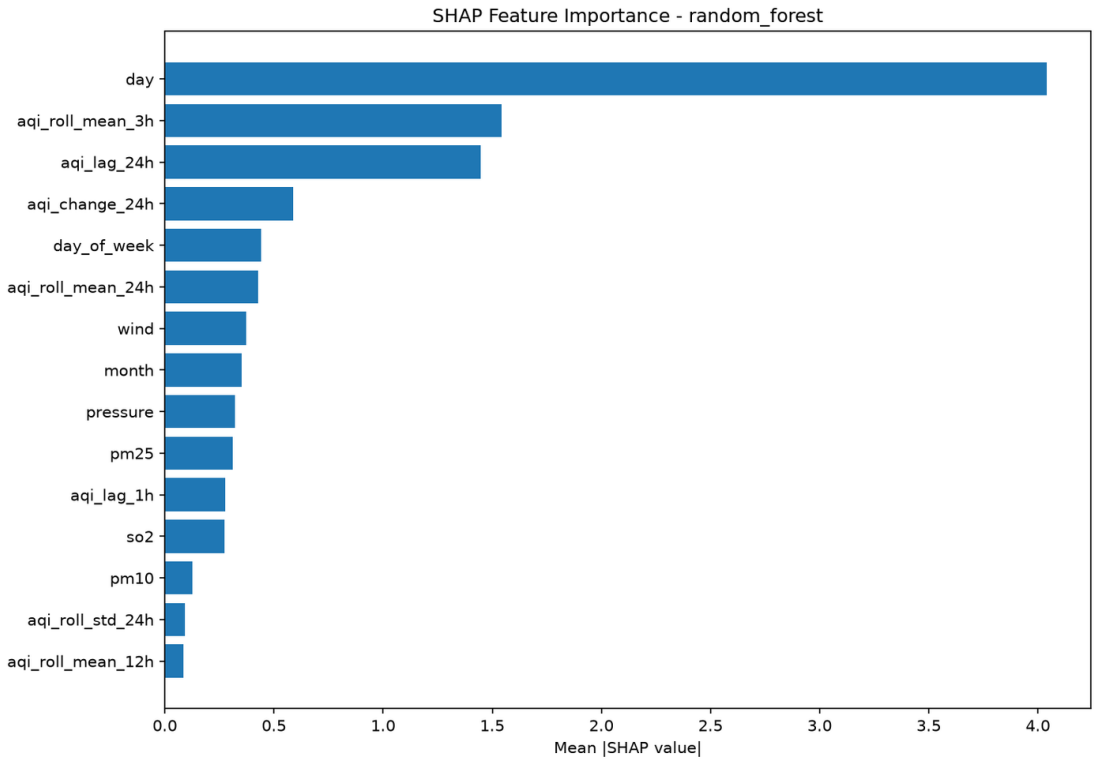
18. Explainability with SHAP

SHAP was incorporated to provide feature-level explanations of model predictions. This is important because a numerical prediction alone does not show which inputs influenced the result.

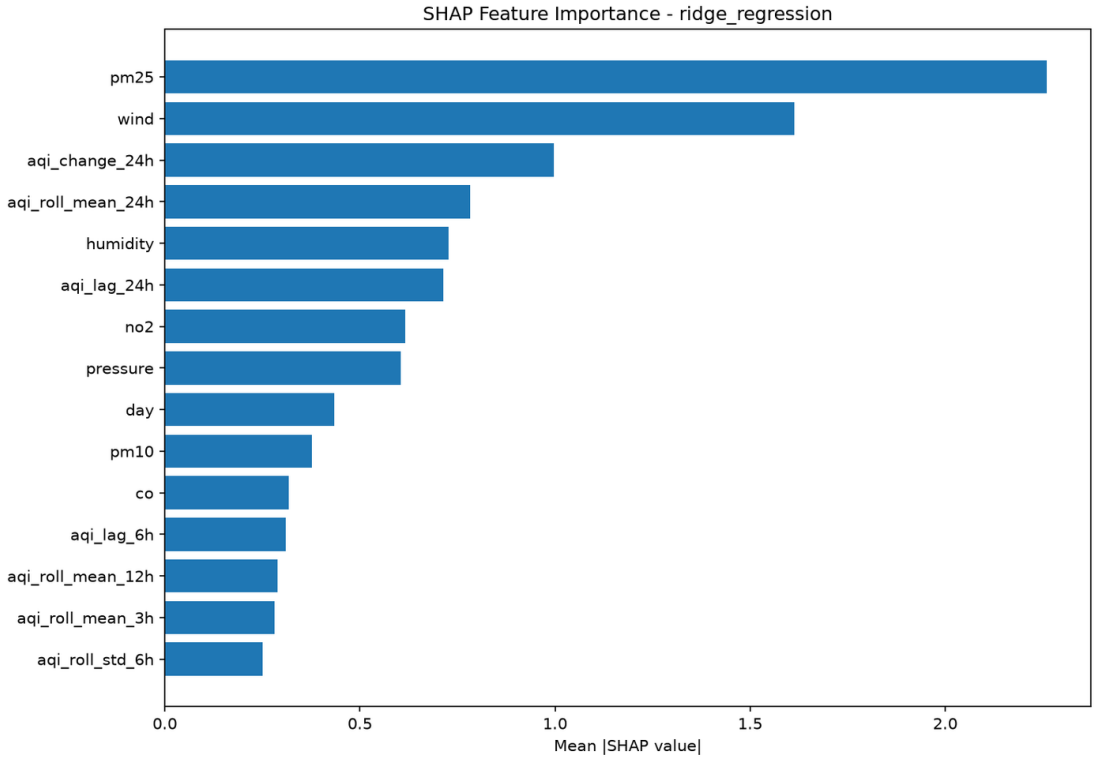
For the production Random Forest, SHAP explanations can be used to show how recent AQI history, pollutant variables, weather conditions, and calendar features contribute to individual predictions. The analysis reinforces the importance of historical AQI features, particularly the previous-hour AQI.

The report should distinguish explainability artifacts from production performance claims. SHAP analyses for other candidate models can be shown as experimentation/explainability material, but the production performance metrics reported in this document belong to the deployed Random Forest model.

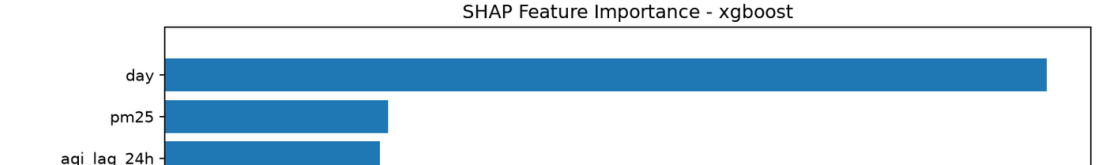
Random Forest



Ridge Regression



XGBoost



19. Deployment

The final application is split across managed services. The FastAPI prediction backend is deployed on Render, while the Streamlit dashboard is deployed on Streamlit Community Cloud. GitHub hosts the source repository and scheduled workflows, and Hopsworks hosts the Feature Store and Model Registry.

Deployment required separating dependency environments. Hopsworks introduced a comparatively heavy and compatibility-sensitive dependency stack, while the public dashboard required a lighter application environment. Separate requirements files were therefore maintained for the application, dashboard, API, and CI/Hopsworks workflows.

The deployed system was tested end-to-end: the public API returned current and next-hour prediction data, and the public Streamlit dashboard successfully displayed the returned values, forecast, metrics, and explainability sections.

20. Engineering Challenges and Solutions

Hopsworks on Windows: Hopsworks dependencies produced compatibility and installation conflicts. A dedicated Conda environment named hopsenv was used for Hopsworks-related operations, keeping those dependencies separate from the main application environment.

Feature Store materialization: newly inserted Feature Store rows were not always immediately available. The live prediction architecture was changed so that Hopsworks remains the training-data system of record while live feature construction uses locally available historical context plus current environmental data.

Deployment dependencies: the initial application dependency stack was too heavy for the dashboard deployment environment. Requirements were separated into lighter application/API/dashboard files and a CI/Hopsworks stack.

Live AQI freshness: the generic AQICN Karachi feed was found to return a stale timestamp during testing. The final live path therefore uses Open-Meteo current AQI and weather data and labels the result as an estimate.

Recursive forecasting: the 72-hour forecast can become relatively stable because the model strongly depends on previous AQI. The limitation is documented rather than hidden, and dedicated multi-horizon forecasting is identified as a future improvement.

21. Limitations

The first limitation is forecast horizon. The production model is validated strongly for the next hour, while recursive performance degrades substantially at 24, 48, and 72 hours.

The second limitation is forecast stability. Because previous AQI is highly influential, recursively generated values can converge toward a relatively stable range instead of representing all real atmospheric variability.

The third limitation is geographic coverage. The current system represents Karachi using a single geographic coordinate rather than a network of physical monitoring stations. Air quality can vary substantially across the city, so additional monitoring locations would improve geographic accuracy.

The fourth limitation is live AQI representation. The deployed API uses an Open-Meteo AQI estimate rather than a direct measurement from a physical Karachi monitoring station.

The fifth limitation is dependence on external services. API outages, stale data, changes in coverage, or service constraints can affect live predictions and dashboard availability.

22. Future Improvements

Several improvements would strengthen the system.

First, dedicated 24-hour, 48-hour, and 72-hour models could replace recursive application of the next-hour model. This would allow each model to learn horizon-specific patterns and reduce recursive error propagation.

Second, the historical dataset could be expanded and additional Karachi monitoring locations could be incorporated. This would improve both statistical robustness and geographic coverage.

Third, prediction intervals could be added so users receive uncertainty information rather than only point forecasts.

Fourth, additional explanatory variables such as traffic indicators, satellite observations, or smoke/fire information could be incorporated.

Fifth, automated forecast-accuracy monitoring could compare future predictions with subsequently available AQI data and trigger model-quality alerts.

Finally, stronger anomaly detection and incoming-data validation could improve resilience against unusual or malformed API responses.

23. Conclusion

Pearls AQI Predictor evolved from a forecasting exercise into a complete machine-learning and MLOps system. The final implementation connects data acquisition, preprocessing, feature engineering, cloud feature management, model training, evaluation, model registration, automated workflows, API serving, recursive forecasting, explainability, and an interactive dashboard.

The Random Forest production model provides strong next-hour performance on the chronological held-out test set, achieving MAE 0.530, RMSE 0.714, and R^2 0.994. At the same time, longer-horizon validation shows substantial degradation when the model is recursively applied to 24–72 hours. Reporting both outcomes provides a more honest assessment of the system.

The most important result of the project is therefore not only the model score. It is the demonstration of a complete lifecycle in which data freshness, reproducibility, deployment, automation, explainability, and limitations are treated as first-class engineering concerns. The system provides a practical foundation for future multi-horizon and geographically distributed AQI forecasting in Karachi.

24. References and Project Resources

Project repository:
https://github.com/umerkhan791/Pearls-AQI-Predictor

Live dashboard:
https://pearls-aqi-predictor-karachicity.streamlit.app/

Prediction API:
https://pearls-aqi-api-xc81.onrender.com/

Primary environmental data source:
Open-Meteo — historical and current air-quality/weather APIs.

Alternative/explored air-quality source:
AQICN / World Air Quality Index API.

Feature and model management:
Hopsworks Feature Store and Model Registry.

Application and automation:
FastAPI, Uvicorn, Streamlit Community Cloud, Render, GitHub Actions.

All numerical model results in this report are from the project's documented final evaluation artifacts and should be interpreted within the scope of the dataset and validation methodology described above.